SIHUL — Metodología de trabajo con GitHub

Repositorio	`winsignares/SIHUL` (un solo repositorio: backend, chatbot, frontend y despliegue)
Ramas permanentes	`develop` (pruebas) y `main` (producción)
Equipo	2 personas: cada cambio lo revisa la otra

1. Objetivo

Que ningún cambio llegue a pruebas ni a producción sin haber sido **probado automáticamente y revisado por otra persona**, y que nadie pueda dañar esas ramas por un error o un atajo.

2. Flujo

```
rama de trabajo ──PR──► develop ──PR──► main
(feature/ fix/ …)       (pruebas)       (producción)
```

Paso	Qué se hace	Quién
1	Crear una **rama nueva** desde `develop` con un prefijo válido	Autor
2	Subir la rama y abrir un *pull request* hacia `develop`	Autor
3	Esperan los controles automáticos; la otra persona revisa y aprueba	Revisor
4	Se fusiona con **squash** (un solo commit limpio) y la rama se borra sola	Autor
5	Para publicar: *pull request* de `develop` a `main`, aprobado por la otra persona; se fusiona con **merge commit**	Ambos
6	Se despliega: `develop` va al entorno de pruebas y `main` a producción	Equipo

Prefijos de rama permitidos: `feature/`, `fix/`, `chore/`, `docs/`, `refactor/` y `hotfix/`. **Mensajes de commit:** `tipo(ámbito): resumen`, por ejemplo `feat(chatbot): responsables por chatbot`.

3. Reglas que aplica GitHub automáticamente

Regla	`develop`	`main`
Prohibido subir cambios directamente (solo por *pull request*)	Sí	Sí
Aprobación de **otra** persona (el autor no puede aprobar el suyo)	1	1
Si se suben commits después de aprobar, hay que aprobar de nuevo	Sí	Sí
El último commit lo debe aprobar alguien distinto de quien lo subió	—	Sí
Conversaciones del *pull request* resueltas antes de fusionar	Sí	Sí
Controles automáticos obligatorios	Sí	Sí
Forma de fusionar	Squash	Merge commit
Prohibido borrar la rama y forzar cambios (`force push`)	Sí	Sí
Excepciones para administradores	Ninguna	Ninguna

Origen permitido de los *pull request*: a `main` solo desde `develop` o `hotfix/*`; a `develop` solo desde las ramas con prefijo válido. Un *pull request* con otro origen es rechazado automáticamente.

4. Controles automáticos (obligatorios para fusionar)

Control	Qué verifica
`check`	Que la rama de origen y el destino respeten el flujo
`backend-tests`	Pruebas del backend sobre una base PostgreSQL real, comprobación de Django y de migraciones pendientes
`frontend-build`	Que el frontend compile

Además se ejecuta una **auditoría de dependencias** (Python y Node) que avisa de vulnerabilidades conocidas. No es obligatoria para fusionar porque aparecen vulnerabilidades nuevas sin que el equipo haya tocado nada; se revisa y se atiende cada semana.

5. Urgencias en producción (hotfix)

1. Crear `hotfix/descripcion` desde `main`.
2. *Pull request* hacia `main`, con las mismas aprobaciones y controles.
3. Después de publicar, llevar el mismo cambio a `develop` para que no se pierda.

No existe un atajo: si hace falta una excepción real, un administrador desactiva la regla de forma temporal y queda registrado.

6. Cómo se relaciona con el despliegue

- El servidor **no usa `git pull`**: cada versión fusionada se construye como imagen y se despliega con un script que exige una copia de seguridad reciente y puede deshacerse.
- Primero la fase manual; cuando las reglas estén aplicadas, al fusionar a `develop` se podrá desplegar solo al entorno de pruebas, y a `main` con aprobación previa.

7. Estado actual

Elemento	Estado
Controles automáticos (flujo de ramas, pruebas, compilación, auditoría)	**Hechos y funcionando**
Asignación automática de revisor, plantilla de *pull request*	**Hechas**
Reglas de protección de `develop` y `main`	**Preparadas; falta que un administrador de la organización las active** (hay una guía con los comandos)
Segunda persona del equipo como revisora	Falta agregar su usuario como revisor del repositorio

Mientras las reglas no estén activas, el flujo se cumple por acuerdo del equipo, pero GitHub aún no lo impide.